

네트워크 4 - 소켓

남궁명수

[Socket]

소켓은 애플리케이션이 운영체제의 네트워크 기능을 사용하기 위한 통신 끝점이다.

프로세스는 네트워크 카드나 TCP/IP 패킷을 직접 제어하지 않고, 소켓을 통해 커널의 네트워크 스택을 사용한다.



쉽게 말하면, 소켓은 프로세스와 운영체제의 네트워크 스택을 연결하는 통신 창구다.

[1] [소켓의 역할]

웹 브라우저가 웹 서버와 통신한다고 생각해보자.

브라우저 프로세스는 다음 정보가 필요하다.

- 어떤 프로토콜을 사용할 것인가?
- 어느 서버와 통신할 것인가?
- 서버의 어느 프로세스와 통신할 것인가?
- 송수신 데이터를 어디에 저장할 것인가?
- 현재 연결 상태는 무엇인가?

운영체제는 이러한 정보를 소켓과 관련된 커널 자료구조에서 관리한다.

애플리케이션은 소켓 API를 호출한다.

```
socket();
bind();
listen();
accept();
connect();
send();
recv();
close();
```

커널은 실제로 다음 작업을 처리한다.

- TCP 연결 설정과 종료
- 포트 관리
- 패킷 생성과 분할
- 시퀀스 번호와 ACK 관리
- 재전송
- 흐름 제어와 혼잡 제어
- 송수신 버퍼 관리
- IP 라우팅
- 네트워크 장치로 패킷 전달

즉, 애플리케이션이 직접 SYN 패킷을 만드는 것이 아니다.

애플리케이션: `connect()` 호출

커널 TCP 스택: 3-Way Handshake 수행

[2] [소켓의 기본 구조]

소켓을 생성할 때는 주로 다음 세 가지 정보를 지정한다.

```
socket(domain, type, protocol);
```

예를 들어, IPv4 TCP 소켓은 다음처럼 생성한다.

```
int fd = socket(AF_INET, SOCK_STREAM, 0);
```

IPv4 UDP 소켓은 다음과 같다.

```
int fd = socket(AF_INET, SOCK_DGRAM, 0);
```

[Domain: 주소 체계]

통신에서 어떤 종류의 주소를 사용할지 지정한다.

Domain	의미
AF_INET	IPv4
AF_INET6	IPv6
AF_UNIX	같은 시스템 내부의 프로세스 간 통신

AF_UNIX 소켓은 네트워크를 거치지 않고 같은 컴퓨터에서 프로세스 간 통신에 사용된다.

[Type: 통신 방식]

Type	의미
SOCK_STREAM	바이트 스트림 방식, 일반적으로 TCP
SOCK_DGRAM	데이터그램 방식, 일반적으로 UDP
SOCK_RAW	IP 패킷을 직접 다루는 Raw Socket

[Protocol: 구체적인 프로토콜]

일반적으로 0을 지정하면 운영체제가 Domain과 Type에 맞는 기본 프로토콜을 선택한다.

```
socket(AF_INET, SOCK_STREAM, 0);  
// 일반적으로 IPv4 TCP 소켓  
  
socket(AF_INET, SOCK_DGRAM, 0);  
// 일반적으로 IPv4 UDP 소켓
```

명시적으로 다음 값을 사용할 수도 있다.

```
IPPROTO_TCP  
IPPROTO_UDP
```

[3] [소켓 주소 구조체]

IPv4 소켓에서는 주소와 포트를 표현하기 위해 sockaddr_in 구조체를 사용한다.

```
struct sockaddr_in {  
    sa_family_t    sin_family; // 주소 체계: AF_INET  
    in_port_t      sin_port;   // 포트 번호  
    struct in_addr  sin_addr;   // IPv4 주소  
};
```

사용 예시는 다음과 같다.

```
struct sockaddr_in server_addr;  
  
server_addr.sin_family = AF_INET;  
server_addr.sin_port = htons(8080);  
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

각 필드의 의미는 다음과 같다.

```
sin_family = AF_INET
→ IPv4 주소를 사용

sin_port = htons(8080)
→ 8080번 포트를 네트워크 바이트 순서로 저장

sin_addr = INADDR_ANY
→ 이 컴퓨터가 가진 모든 IPv4 인터페이스에서 연결을 받음
```

[호스트 바이트 순서와 네트워크 바이트 순서]

네트워크에서는 바이트 순서로 Big Endian을 사용한다.

따라서 정수 형태의 포트와 IP 주소를 네트워크 바이트 순서로 변환해야 한다.

함수	의미
htons()	Host to Network Short
htonl()	Host to Network Long
ntohs()	Network to Host Short
ntohl()	Network to Host Long

예를 들어:

```
server_addr.sin_port = htons(8080);
```

은 호스트의 16비트 정수 8080을 네트워크 바이트 순서로 변환한다.

[일반적인 컴퓨터는 리틀 엔디안, 네트워크는 빅 엔디안]

```
일반적인 컴퓨터(x86, ARM) → Little Endian
네트워크 바이트 순서      → Big Endian
```

예를 들어, 16진수 0x12345678을 메모리에 저장하면:

```
빅 엔디안    : 12 34 56 78
리틀 엔디안  : 78 56 34 12
```

- 빅엔디안: 큰 자리 바이트부터 저장
- 리틀 엔디안: 작은 자리 바이트부터 저장

네트워크에서는 서로 다른 CPU를 사용하는 컴퓨터끼리도 통신해야 하므로, 다중 바이트 숫자를 빅 엔디안으로 보내기로 표준화한다.

이를 Network Byte Order라고 한다.

그래서 리틀 엔디안 컴퓨터는 포트 번호 같은 값을 전송하기 전에 변환한다.

```
htons(port);    // host → network, 16비트
htonl(value);   // host → network, 32비트
ntohs(port);    // network → host, 16비트
ntohl(value);   // network → host, 32비트
```

[4] [소켓과 파일 디스크립터]

Linux에서는 소켓을 생성하면 정수 형태의 파일 디스크립터가 반환된다.

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

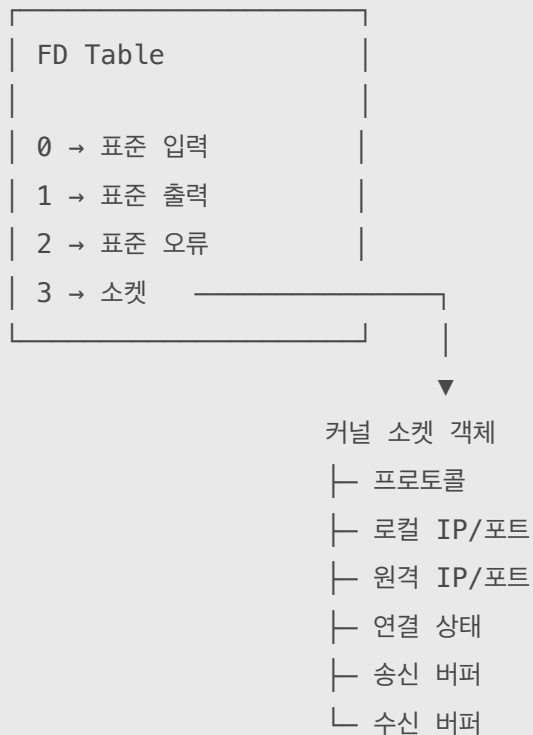
예를 들어 socket()이 3을 반환하면:

```
sockfd = 3
```

프로세스는 이후 소켓 객체를 직접 다루지 않고 파일 디스크립터를 사용한다.

```
connect(sockfd, ...);
write(sockfd, ...);
read(sockfd, ...);
close(sockfd);
```

개념적으로는 다음과 같다.
프로세스



따라서 Linux에서는 소켓에도 다음 시스템 콜을 사용할 수 있다.

```
read();
write();
close();
```

하지만 “소켓이 실제 디스크 파일이다”라는 뜻은 아니다.
Linux가 파일과 소켓 같은 커널 객체를 파일 디스크립터라는 공통 인터페이스로 다룬다는 의미다.

[5] [소켓이 가지는 통신 정보]

TCP 연결은 일반적으로 다음 네 가지 값으로 구분한다.

```
출발지 IP
출발지 Port
목적지 IP
목적지 Port
```

이를 **4-Tuple**이라고 한다.
예를 들어:

```
192.168.0.10:53000 → 142.251.119.100:80
```

각 정보는 다음을 의미한다.

```
클라이언트 IP   = 192.168.0.10
클라이언트 포트 = 53000
서버 IP         = 142.251.119.100
서버 포트       = 80
```

프로토콜까지 포함해 다음처럼 구분하기도 한다.

```
Protocol
Source IP
Source Port
Destination IP
Destination Port
```

이를 **5-Tuple**이라고 한다.

```
TCP, 192.168.0.10, 53000, 142.251.119.100, 80
```

[6] [서버 소켓과 클라이언트 소켓]

TCP 서버와 클라이언트의 가장 큰 차이는 다음과 같다.

```
서버: 특정 포트에서 연결 요청을 기다린다.
클라이언트: 서버의 IP와 포트로 연결을 요청한다.
```

전체 흐름은 다음과 같다.

TCP 서버		TCP 클라이언트
socket()		socket()
bind()		
listen()		
		connect()
accept() <— 3-Way Handshake	—	
read()/write() ← 데이터 통신 →		write()/read()
close() ← 연결 종료 →		close()

[7] [TCP 서버 동작 과정]

TCP 서버는 일반적으로 다음 순서로 동작한다.

```
socket();  
bind();  
listen();  
accept();  
read();  
write();  
close();
```

[1단계: socket()]

TCP 소켓을 생성한다.

```
int listen_fd = socket(AF_INET, SOCK_STREAM, 0);
```

성공하면 파일 디스크립터를 반환한다.

```
listen_fd = 3
```

이 시점에는 아직 IP 주소나 포트가 지정되지 않았고 클라이언트와 연결되지도 않았다.

[2단계: bind()]

소켓에 서버가 사용할 로컬 IP 주소와 포트 번호를 지정한다.

```
bind(listen_fd, (struct sockaddr *)&server_addr,  
      sizeof(server_addr));
```

예를 들어:

```
IP    = 0.0.0.0  
Port = 8080
```

0.0.0.0 또는 INADDR_ANY는 서버가 가진 모든 IPv4 네트워크 인터페이스에서 연결을 받겠다는 의미다.

```
127.0.0.1:8080
```

→ 같은 컴퓨터에서만 접근 가능

```
0.0.0.0:8080
```

→ 서버의 모든 IPv4 인터페이스를 통해 접근 가능

```
192.168.0.10:8080
```

→ 해당 IP가 설정된 인터페이스를 통해서만 접근 가능

bind()는 서버가 어떤 주소에서 요청을 받을지 지정하는 과정이다.

[3단계: listen()]

소켓을 클라이언트 연결 요청을 기다리는 **리스닝 소켓**으로 변경한다.

```
listen(listen_fd, backlog);
```

예를 들어:

```
listen(listen_fd, 128);
```

이제 커널은 해당 소켓을 통해 들어오는 TCP 연결 요청을 관리한다.

```
일반 TCP 소켓
    ↓ listen()
리스닝 소켓
```

backlog는 단순히 “동시에 접속할 수 있는 전체 클라이언트 수”가 아니다.
아직 애플리케이션이 accept()하지 않은 연결 요청들을 커널이 대기시킬 수 있는 큐의 크기와 관련된 값이다.

실제 세부 동작은 운영체제 구현과 설정에 따라 달라질 수 있다.

[4단계: accept()]

연결된 클라이언트를 하나 꺼내 처리할 새로운 소켓을 반환한다.

```
int client_fd = accept(listen_fd, ...);
```

여기가 매우 중요하다.

서버에는 서로 다른 역할을 하는 소켓이 존재한다.

```
listen_fd
→ 새로운 연결 요청을 계속 받는 소켓

client_fd
→ 특정 클라이언트와 실제 데이터를 통신하는 소켓
```

예를 들어 accept()가 새로운 파일 디스크립터 4를 반환했다면:

```
FD 3: 리스닝 소켓
FD 4: 클라이언트 A와 연결된 소켓
```

다른 클라이언트가 접속하면 다시 새로운 연결 소켓이 반환된다.

```
FD 3: 리스닝 소켓
FD 4: 클라이언트 A 연결
FD 5: 클라이언트 B 연결
FD 6: 클라이언트 C 연결
```

구조는 다음과 같다.

```
└─ client_fd 4 - 클라이언트 A
listen_fd 3 - accept └─ client_fd 5 - 클라이언트 B
                     └─ client_fd 6 - 클라이언트 C
```

리스닝 소켓은 계속 새로운 연결을 받고, 연결 소켓은 각각의 클라이언트와 데이터를 주고받는다.

[accept()와 3-Way Handshake의 관계]

흔히 accept()가 직접 3-Way Handshake를 수행한다고 생각하기 쉽지만, 정확하게는 커널의 TCP 스택이 Handshake를 처리한다.

```
클라이언트 connect()
    |
    ▼
커널이 3-Way Handshake 수행
    |
    ▼
연결 완료
    |
    ▼
서버 accept()가 완료된 연결을 꺼내 반환
```

accept()는 일반적으로 커널이 연결 처리를 완료한 클라이언트 연결을 애플리케이션이 가져오는 시스템 콜이다.

연결된 클라이언트가 없다면 기본 Blocking 소켓의 accept()는 연결이 들어올 때까지 대기한다.

[5단계: read()와 write()]

연결된 소켓을 통해 데이터를 주고받는다.

```
read(client_fd, buffer, sizeof(buffer));
write(client_fd, response, response_len);
```

또는 소켓 전용 함수인 다음 함수를 사용할 수 있다.

```
recv();
send();
```

```
read()/recv() → 소켓에서 데이터 읽기
write()/send() → 소켓으로 데이터 쓰기
```

TCP 서버는 리스닝 소켓이 아니라 `accept()`가 반환한 연결 소켓을 통해 데이터를 주고받는다.

```
// 잘못된 개념
read(listen_fd, ...);

// 실제 통신
read(client_fd, ...);
```

[6단계: `close()`]

통신이 끝난 연결 소켓을 닫는다.

```
close(client_fd);
```

TCP 소켓을 정상적으로 닫으면 커널이 FIN과 ACK를 이용한 연결 종료 절차를 수행한다.

서버 자체를 종료하려면 리스닝 소켓도 닫는다.

```
close(listen_fd);
```

일반적으로 서버는 클라이언트 연결 하나를 처리한 뒤에도 리스닝 소켓을 유지한다.

```
client_fd 닫기
→ 해당 클라이언트와 연결만 종료

listen_fd 닫기
→ 새로운 클라이언트 연결을 더 이상 받지 않음
```

[8] [TCP 클라이언트 동작 과정]

TCP 클라이언트는 일반적으로 다음 순서로 동작한다.

```
socket();
connect();
write();
read();
close();
```

[1단계: `socket()`]

TCP 소켓을 생성한다.

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

아직 서버와 연결된 상태는 아니다.

[2단계: connect()]

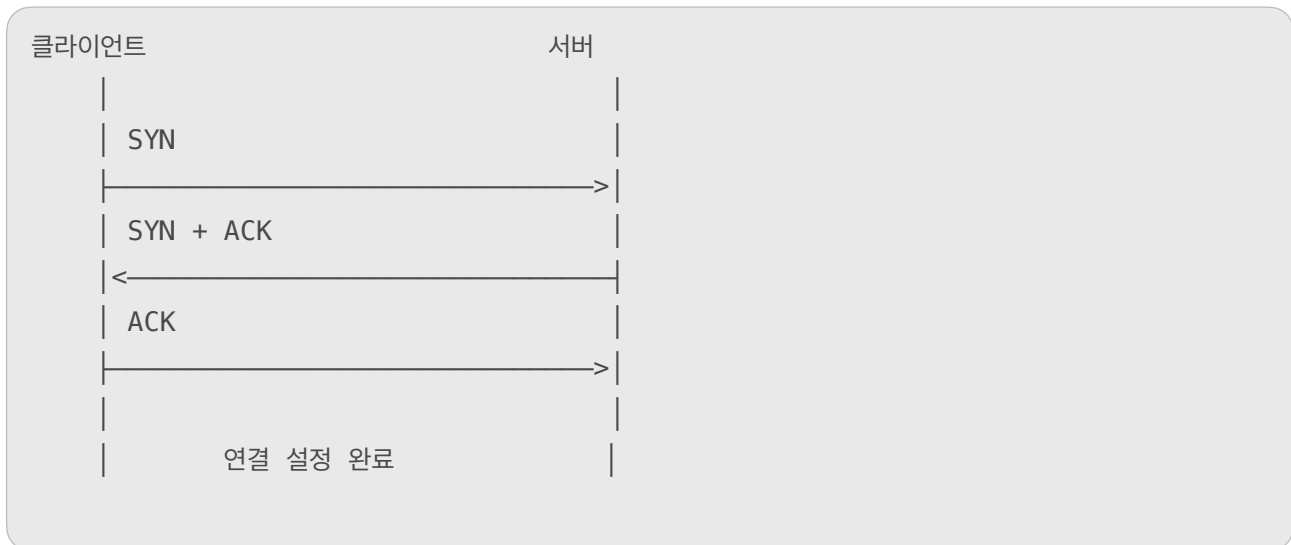
서버의 IP 주소와 포트로 연결을 요청한다.

```
connect(sockfd, (struct sockaddr *)&server_addr,  
        sizeof(server_addr));
```

예를 들어:

```
서버 IP   = 192.168.0.20  
서버 Port = 8080
```

클라이언트가 connect()를 호출하면 커널의 TCP 스택이 3-Way Handshake를 수행한다.



기본 Blocking 소켓에서는 연결이 성공하거나 오류가 발생할 때까지 connect()가 대기할 수 있다.

[클라이언트의 로컬 포트]

클라이언트가 직접 bind()하지 않아도 커널이 임시 포트인 Ephemeral Port를 자동으로 할당한다.

예를 들어:

```
클라이언트: 192.168.0.10:53000  
서버:       192.168.0.20:8080
```

- 8080: 서버가 서비스를 제공하기 위해 고정해서 사용하는 포트
- 53000: 커널이 클라이언트에 임시로 할당한 포트

클라이언트도 특별한 목적이 있다면 connect() 전에 bind()를 호출하여 특정 로컬 IP나 포트를 지정할 수 있다.

[3단계: 데이터 송수신]

연결이 설정되면 데이터를 전송한다.

```
write(sockfd, request, request_len);  
read(sockfd, buffer, sizeof(buffer));
```

TCP는 전이중 통신이므로 클라이언트와 서버 모두 언제든지 데이터를 보내거나 받을 수 있다.

```
클라이언트 —————> 서버  
클라이언트 <———— 서버
```

“클라이언트가 반드시 먼저 쓰고 서버가 반드시 나중에 응답해야 한다”는 규칙은 TCP 자체의 규칙이 아니다. 요청과 응답 순서는 HTTP 같은 애플리케이션 프로토콜이 정한다.

[4단계: close()]

통신이 끝나면 소켓을 닫는다.

```
close(sockfd);
```

커널은 정상적인 TCP 연결 종료 절차를 수행한다.

[9] [TCP 서버의 전체 구조]

TCP 서버는 여러 클라이언트를 계속 받아야 하므로 일반적으로 반복문을 사용한다.

```
int listen_fd = socket(AF_INET, SOCK_STREAM, 0);  
  
bind(listen_fd, ...);  
listen(listen_fd, 128);  
  
while (1) {  
    int client_fd = accept(listen_fd, ...);  
  
    read(client_fd, ...);  
    write(client_fd, ...);  
  
    close(client_fd);  
}  
  
close(listen_fd);
```

하지만 이 코드는 클라이언트를 한 명씩 순차적으로 처리한다.

클라이언트 A 처리 완료

↓

클라이언트 B 처리

↓

클라이언트 C 처리

클라이언트 A의 처리가 오래 걸리면 다른 클라이언트는 기다려야 한다.
여러 클라이언트를 동시에 처리하려면 다음 방식들을 사용할 수 있다.

- 프로세스 생성
- 스레드 생성
- 스레드 풀
- select
- poll
- epoll
- 비동기 I/O
- Go의 Goroutine

select

여러 파일 디스크립터를 동시에 감시하고, I/O 준비된 대상을 알려주는 시스템 콜이다. 매번 전체 대상을 순회하며 개수 제한이 있다.

스레드풀(Thread Pool)

미리 일정 개수의 스레드를 생성해 두고, 작업 큐에 들어오는 작업들을 대기 중인 스레드에 할당하여 처리하는 기법

poll

select처럼 여러 파일 디스크립터를 감시하지만 배열로 관리하여 개수 제한을 개선했다. 다만 매번 전체 배열을 순회해야 한다.

epoll

감시 대상을 커널에 등록하고 이벤트가 발생한 파일 디스크립터를 중심으로 반환하는 리눅스 기능이다.
대규모 네트워크 연결 처리에 효율적이다.

비동기 I/O

I/O 작업을 요청한 뒤 완료될 때까지 기다리지 않고 다른 작업을 수행하는 방식이다.
작업이 끝나면 완료 결과를 전달받는다.

Goroutine: Go 런타임이 관리하는 가벼운 동시 실행 단위이다. 많은 고루틴을 적은 OS 스레드에 나누어 실행할 수 있다.

Go 서버에서는 연결마다 고루틴을 실행하는 구조를 자주 사용한다.

```
for {
    conn, err := listener.Accept()
    if err != nil {
        continue
    }

    go handleConnection(conn)
}
```

리스닝 고루틴

```
|
├─ 클라이언트 A → 고루틴 A
├─ 클라이언트 B → 고루틴 B
└─ 클라이언트 C → 고루틴 C
```

고루틴을 사용하더라도 무제한으로 생성하면 메모리, 파일 디스크립터 등의 자원이 고갈될 수 있으므로 실제 서버에서는 연결 제한, 타임아웃, 세마포어 등을 함께 고려해야 한다.

[10] [UDP 서버와 클라이언트]

UDP에는 연결 설정 과정이 없기 때문에 listen()과 accept()를 사용하지 않는다.

[UDP 서버]

```
socket();
bind();
recvfrom();
sendto();
close();
```

```
socket()
  ↓
bind()
  ↓
recvfrom()로 데이터그램 수신
  ↓
sendto()로 송신자에게 응답
```

recvfrom()은 데이터뿐 아니라 데이터를 보낸 상대방의 주소도 반환한다.

```
recvfrom(  
    sockfd,  
    buffer,  
    sizeof(buffer),  
    0,  
    (struct sockaddr *)&client_addr,  
    &client_len  
);
```

서버는 반환받은 client_addr를 이용해 응답한다.

```
sendto(  
    sockfd,  
    response,  
    response_len,  
    0,  
    (struct sockaddr *)&client_addr,  
    client_len  
);
```

[UDP 클라이언트]

```
socket();  
sendto();  
recvfrom();  
close();
```

UDP는 연결 소켓을 별도로 만들지 않으므로 서버가 하나의 소켓으로 여러 클라이언트의 데이터그램을 받을 수 있다.

```
클라이언트 A ┐  
클라이언트 B ┴──> UDP 서버 소켓  
클라이언트 C ┘
```

각 데이터그램에 포함된 출발지 IP와 포트를 보고 어느 클라이언트가 보냈는지 구분한다.

[11] [Blocking 소켓]

기본적으로 소켓 시스템 콜은 Blocking 방식으로 동작할 수 있다.

[accept()]

완료된 연결이 없다면 클라이언트가 접속할 때까지 기다린다.

```
accept()  
↓  
클라이언트 연결 없음  
↓  
스레드 대기
```

[read() 또는 recv()]

수신 버퍼에 데이터가 없다면 데이터가 도착할 때까지 기다린다.

```
read()  
↓  
수신 데이터 없음  
↓  
스레드 대기
```

단, TCP에서 상대방이 정상적으로 연결을 종료해 FIN을 받고 수신할 데이터가 모두 소진되면 read()는 0을 반환한다.

```
n = read(fd, buffer, sizeof(buffer));  
  
n > 0    // 데이터를 읽음  
n == 0    // 상대방이 정상적으로 송신 방향 종료  
n < 0    // 오류
```

[connect()]

연결이 성공하거나 실패할 때까지 기다릴 수 있다.

이러한 대기를 피하거나 하나의 스레드에서 여러 소켓을 관리하기 위해 Non-blocking 소켓과 epoll 등을 사용한다.

[11] [Go에서의 TCP 서버와 클라이언트]

Go의 net 패키지는 내부적으로 운영체제의 소켓 기능을 사용한다.

[TCP 서버]

```
package main

import (
    "bufio"
    "fmt"
    "net"
)

func main() {
    listener, err := net.Listen("tcp", ":8080")
    if err != nil {
        panic(err)
    }
    defer listener.Close()

    for {
        conn, err := listener.Accept()
        if err != nil {
            continue
        }

        go handleConnection(conn)
    }
}

func handleConnection(conn net.Conn) {
    defer conn.Close()

    reader := bufio.NewReader(conn)

    message, err := reader.ReadString('\n')
    if err != nil {
        return
    }

    fmt.Print("수신: ", message)
    conn.Write([]byte("서버 응답\n"))
}
```

각 부분은 개념적으로 다음 소켓 함수와 연결된다.

```
net.Listen()
→ socket() + bind() + listen()

listener.Accept()
→ accept()

conn.Read()
→ read()/recv()

conn.Write()
→ write()/send()

conn.Close()
→ close()
```

[TCP 클라이언트]

```
package main

import (
    "bufio"
    "fmt"
    "net"
)

func main() {
    conn, err := net.Dial("tcp", "127.0.0.1:8080")
    if err != nil {
        panic(err)
    }
    defer conn.Close()

    _, err = conn.Write([]byte("안녕하세요\n"))
    if err != nil {
        panic(err)
    }

    reader := bufio.NewReader(conn)

    response, err := reader.ReadString('\n')
    if err != nil {
```

```

        panic(err)
    }

    fmt.Print("응답: ", response)
}

```

```

net.Dial()
→ socket() + connect()

conn.Write()
→ write()/send()

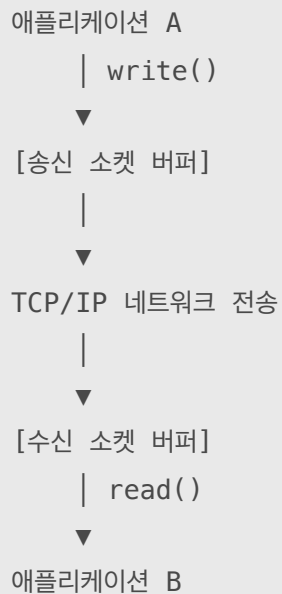
conn.Read()
→ read()/recv()

conn.Close()
→ close()

```

[12] [소켓 버퍼]

커널은 일반적으로 소켓마다 송신 버퍼와 수신 버퍼를 관리한다.



[송신 과정]

애플리케이션이 `write()`를 호출하면 데이터가 반드시 즉시 네트워크로 전송 완료되는 것은 아니다. 일반적으로 데이터는 먼저 커널의 송신 버퍼에 복사되고, 커널의 TCP/IP 스택이 네트워크 상태에 따라 전송한다. 따라서 Blocking 소켓에서 `write()`가 성공했다는 것은 보통 다음 의미다. 데이터가 상대방 애플리케이션까지 전달됐다는 뜻이 아니라, 커널이 전송할 데이터를 받아들였다는 뜻이다.

[수신 과정]

네트워크에서 데이터가 도착하면 커널은 이를 처리하여 소켓의 수신 버퍼에 저장한다. 애플리케이션이 `read()`를 호출하면 커널 수신 버퍼에 있는 데이터를 사용자 공간으로 복사한다.

```
graph TD; A[네트워크 패킷 도착] --> B[NIC 및 커널 네트워크 스택 처리]; B --> C[소켓 수신 버퍼]; C --> D[애플리케이션 read()];
```

네트워크 패킷 도착

↓

NIC 및 커널 네트워크 스택 처리

↓

소켓 수신 버퍼

↓

애플리케이션 `read()`

TCP 흐름 제어의 `rwnd`는 이 수신 버퍼의 남은 공간과 관련된다.